Are Self-Evolving Agent Skills Learning Skills?

Anonymous submission

Abstract

We introduce a lifecycle audit for determining whether score gains from self-evolving agent skills represent retained skill improvement. A post-revision score alone is insufficient because optimizer variation, validation selection, execution randomness, and verifier choice can improve the endpoint without creating a durable artifact effect. We combine repeated evaluations, controlled task variations, and lifecycle interventions to examine whether improvements from skill evolution persist and transfer, and to trace their origins to feedback, revision, selection, skill content, and evaluation. The audit separates five evidential claims: selected gain, retained artifact effect, bounded transfer, evolution-pathway dependence, and measurement dependence. This claim-grounded framework lets self-evolving systems report what their experiments establish without equating a better benchmark score with reusable skill learning.

Introduction

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill (Li and Hu 2026; Liu et al. 2026b; Yang et al. 2026; Xia et al. 2026).

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. These possibilities do not imply that self-evolution is ineffective. They imply that an endpoint score alone cannot locate where the improvement resides.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain?

We organize the experiments around utility, behavioral scope, and lifecycle attribution. Repeated standard-task evaluations of SkillEvolver, SPARK, and CoEvoSkills measure skill utility, and controlled task variants examine how far that utility extends. Inspection of skill content and execution traces then identifies reusable decisions expressed during deployment. Complementary mechanism studies in SkillRevise, SkillHone, and SkillOpt examine retention and the origins of observed gains. These studies represent trace-conditioned revision, history-based iterative honing, and validation-gated bounded editing in frozen-agent systems with persistent external skill artifacts.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The resulting picture is deliberately more qualified than either *self-evolution works* or *self-evolution only exploits the evaluator*. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.
- **Lifecycle audit.** We apply matched interventions across

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

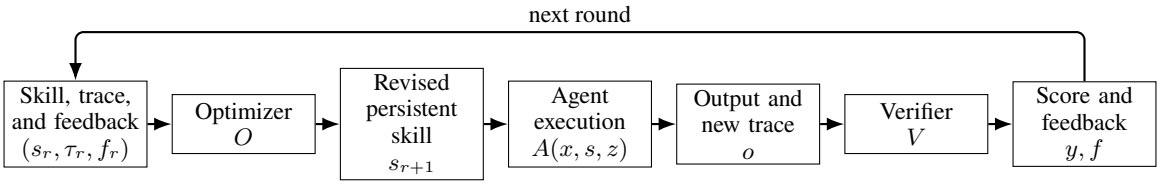


Figure 1: Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

three representative skill-evolution mechanisms without turning heterogeneous studies into a method leaderboard.

- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

Problem Formulation

Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The following experiments address these questions in turn.

Skill Utility and Reuse on SkillsBench

We first ask whether completed skills help on their source tasks, then whether that utility extends when the tasks change. Repeated standard-task evaluations establish the observed benefit of deploying the final artifacts. Controlled variants examine their scope of reuse, and inspection of skill content and execution traces connects these outcomes to the decisions made during deployment.

Experimental Setup

We evaluate SkillEvolver (Zhang et al. 2026a), SPARK (Zhou et al. 2026b), and CoEvoSkills (Zhang et al. 2026b) on SkillsBench (Li et al. 2026), with two runs per method. Each run pairs an execution using the generated or evolved skill with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

We also count tasks with positive, negative, and zero differences.

Do Completed Skills Help on Their Source Tasks?

All three methods achieve positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, with 27 versus six improved/regressed tasks in run 1 and 26 versus three in run 2. CoEvoSkills gains 10.24 and 9.76 points, with 17 improved tasks in each run and nine versus ten regressions. For these two methods, the paired gains remain similar even though absolute baseline rewards differ between runs.

SPARK has smaller positive gains of 1.84 and 6.45 points. Its first run has nearly balanced improvements and regressions (nine versus eight); the second has 13 improvements and five regressions. Thus, observing a positive mean gain twice can accompany different patterns of task-level benefit.

The repeated positive direction establishes a consistent aggregate pattern across the two runs for each method. The coexistence of improvements, regressions, and ties also shows that skill utility varies across tasks. These paired comparisons establish the observed source-task utility of the completed artifacts. They provide the starting point for the reuse question:

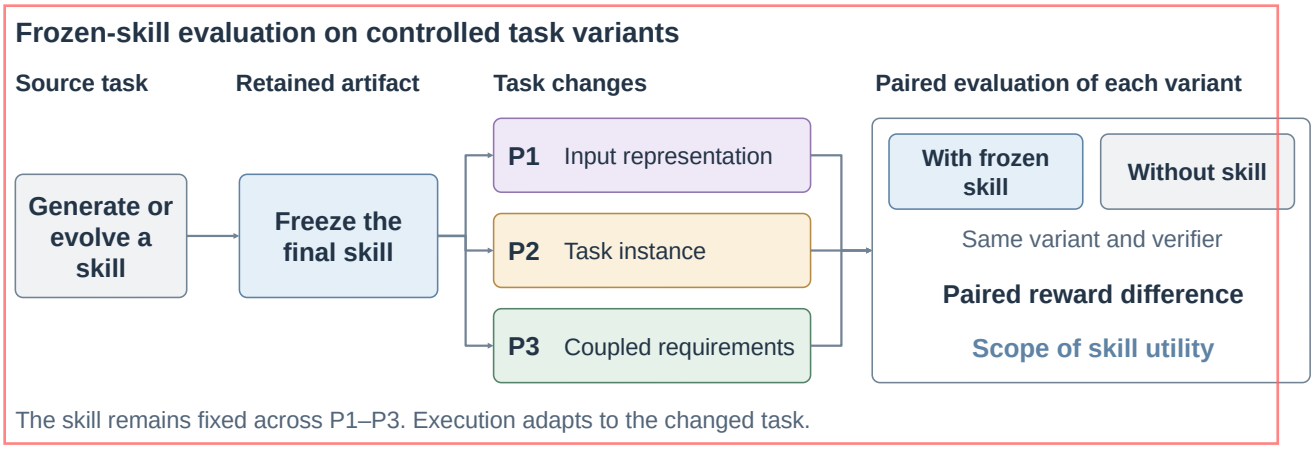


Figure 2: Evaluation of skill reuse under task variation. A source-task skill is frozen and reused under representation (P1), instance (P2), and requirement (P3) changes. Comparison with a no-skill execution of the same variant measures the completed artifact's utility.

Method	Run	Tasks	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

Table 1: Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

whether the content that helps on a source task remains useful when its inputs, instance, or requirements change.

Do Evolved Skills Remain Useful When Tasks Change?

A source-task skill may capture a reusable procedure, a decision rule, or assumptions specific to that instance. We deploy these skills on controlled variants while keeping each artifact frozen (Figure 2). The executor interprets the changed task and decides how to use the existing skill. Its reward difference from a no-skill execution of the same variant measures the final artifact's utility under that change.

Three forms of task change. Input-representation changes (P1) alter how the task is presented, including names, column arrangements, or schemas. They test whether the executor can recover the relevant roles of the inputs while using the skill. Task-instance changes (P2) replace the data or instance structure. They test whether the procedure can be instantiated with new values rather than relying on a solution specific to the source instance. Coupled-requirement changes (P3) alter or add requirements that must be handled together. They test whether the executor can adapt the procedure while preserving the constraints that determine a valid output. These changes probe different aspects of reuse rather than successive levels of difficulty.

The record contains 222 task variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills. Panel B separates the within-variant differences by type of change.

Overall utility under task variation. SkillEvolver increases mean reward from 41.87 to 53.81, a gain of 11.94 points, with 47 improved and 20 regressed variants. Thus, its source-task skills retain positive average utility after the tasks change. SPARK increases mean reward from 41.61 to 42.25, a gain of 0.65 points, with 32 improvements and 37 regressions. The positive mean alongside more regressions than improvements reflects the magnitudes of the reward changes: the mean and the number of benefited variants capture different properties. CoEvoSkills decreases mean reward from 41.61 to 36.60, a change of −5.01 points, with 26 improvements and 41 regressions. Its positive source-task gains therefore coexist with negative average utility on the recorded variants.

Where does utility extend? SkillEvolver has positive gains under every type of change: 11.85 points on P1, 14.56 on P2, and 9.72 on P3. Improved variants also outnumber regressed variants within each type: 15 versus seven, 15 ver-

A. Overall utility on task variants

Method	Variants	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	222	41.87	53.81	+11.94	47	20	155
SPARK	221	41.61	42.25	+0.65	32	37	152
CoEvoSkills	221	41.61	36.60	−5.01	26	41	154

B. Utility by type of task change

Method	P1: Representation		P2: Instance		P3: Requirements	
	N	Δ	N	Δ	N	Δ
SkillEvolver	77	+11.85	68	+14.56	77	+9.72
SPARK	77	+2.00	68	−5.23	76	+4.53
CoEvoSkills	77	−3.59	68	−9.51	76	−2.42

Table 2: Skill utility under task variation: descriptive summary of historical and recovery evaluations. Panel A reports overall outcomes; Panel B separates representation, instance, and requirement changes. Each method is paired with its own same-variant no-skill baseline. Scores and within-variant differences use the scale in Table 1.

sus six, and 17 versus seven, respectively. Its overall gain consequently spans all three forms of variation. The largest mean gain occurs under P2, where applying the source skill requires working with a changed instance.

The other methods exhibit different profiles. SPARK remains positive on P1 and P3, at 2.00 and 4.53 points, but falls by 5.23 points on P2. CoEvoSkills has negative average changes on all three types, with its largest decline also on P2 (-9.51 points, compared with -3.59 on P1 and -2.42 on P3). Instance changes thus separate the observed reuse profiles especially clearly: SkillEvolver's largest positive gain and the other methods' largest declines occur in the same category of variation.

What the combined results reveal. All three methods obtain positive mean gains in both standard-task runs, yet their variant profiles differ substantially. Success on the source tasks and usefulness after task changes are distinct empirical properties. SkillEvolver provides the clearest positive reuse pattern in these records, whereas SPARK's benefit depends on the type of change and CoEvoSkills' source-task benefit does not extend on average. Approximately seven in ten variants remain tied with the baseline for each method. The gains therefore arise from a subset of changed outcomes, motivating a closer examination of the decisions expressed when a skill helps or hinders execution.

What Is Preserved When a Skill Is Reused?

To interpret the variant results, we inspect the skill artifacts together with their execution traces. We distinguish the procedure stored in a skill from the way an executor applies it to a new task. A script can preserve an action sequence, while successful reuse also depends on recognizing when that sequence applies, adjusting its inputs, and checking its output. Figure 3 organizes the inspected evidence around these decisions.

Diagnostic rules guide adaptation. In the inspected SkillEvolver artifacts, intermediate checks guide the choice and revision of a procedure. Candidate discovery, diagnosis, and refinement appear as separate operations, with conditions explaining when to revisit an intermediate result. This structure gives the executor opportunities to respond to changed inputs before committing to an output. In the signal-detection traces, for example, the executor uses diagnostic checks to distinguish competing candidate explanations and adjusts the search accordingly. The reusable content is the rule for evaluating a candidate; the candidate values and search settings can change with the task.

Correctness constraints survive solution changes. A second form of reusable content specifies which properties the output must preserve. The inspected scheduling skill expresses constraints relative to the supplied instance and baseline, allowing the executor to construct a different schedule while checking the same repair requirements. Failure traces make this distinction visible: much of a workflow can execute correctly while one omitted requirement invalidates the final result. This helps explain why preserving a familiar procedure is insufficient when a variant changes how inputs are represented or how requirements interact.

Reuse combines preserved rules with new execution. Across the inspected cases, successful adaptation preserves decision criteria and correctness checks while changing input handling, search choices, or the constructed solution. This illustrates how a transferred artifact can help the executor decide what to retain and what to recompute. SkillEvolver's positive profile across P1–P3 is consistent with this form of reuse. The trace evidence identifies concrete instances of that behavior. Complementary interventions on SkillRevise, SkillHone, and SkillOpt later in the paper test component effects within their respective artifact lineages.

The variant evaluations characterize how far final-skill utility extends beyond the source task. The following retention and lifecycle studies examine whether improvements persist under fresh deployment and which parts of the evolution process produce them.

Identifying Retained Skill Effects

Evaluation Design and Contrasts

We analyze three persistent-skill systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles (Liu et al. 2026b).

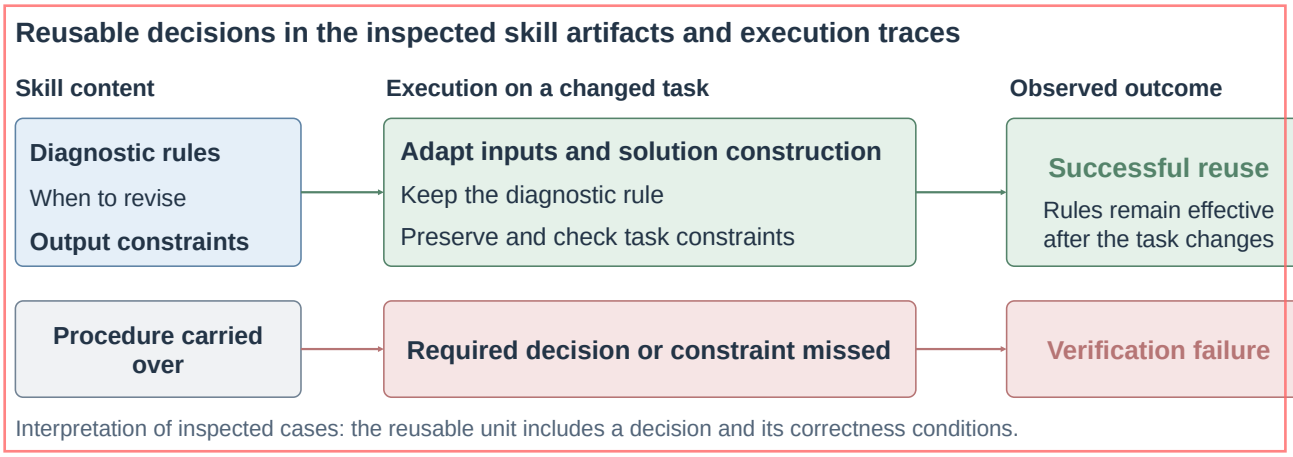


Figure 3: Skill reuse in the inspected cases. Diagnostic rules guide adaptation, and correctness constraints guide output checks. Successful reuse preserves these decisions while adapting the solution; unsuccessful reuse can carry over a procedure while missing a required decision or constraint.

SkillHone accumulates practice history and iteratively reflects on the persistent skill (Li and Hu 2026). SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption (Yang et al. 2026). The systems are studied as representative mechanisms, not as directly comparable leaderboard entries.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 3).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.

Panel and contrast	Main result	Supported inference and boundary
Panel A: Retention under fresh deployment		
Historical pass → fresh repeats	12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone	A selected pass is not a deployment probability; denominators differ and are not a method ranking.
SkillRevise final → no-skill	16/48 vs. 14/48; $+4.2$ pp, 95% CI $[-10.4, 20.8]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
SkillHone final → no-skill	12/48 vs. 14/48; -4.2 pp, 95% CI $[-20.8, 14.6]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
Panel B: Revision effects under controlled variations		
Invoice parent → final	$0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$	Stable but narrow same-contract transfer.
Energy parent → final	Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$	Directional parameter transfer in a small stochastic case.
SkillOpt SearchQA parent → final	Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails	Local held-out gain, not stable general procedure learning.

Table 3: Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

Attributing Observed Skill Gains

Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

Figure 4 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

Revision and Selection Pathways

These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

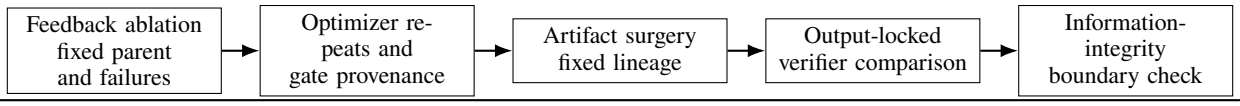
Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let P denote the complete parent, F the complete final artifact, $P + B$ the parent with component B, and $F - B$ the final artifact with that component removed.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, SkillRevise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied



Lifecycle stage	SkillRevise	SkillHone	SkillOpt
Feedback and revision	Diagnosis ablation; revision traces	Feedback-view ablation	Feedback branches; optimizer repeats
Selection and adoption	Revision history	Iterative honing history	Native validation gate; accepted and rejected candidates
Artifact mediation	Not yet matched at the same strength	Invoice and Energy surgery	SearchQA component surgery
Measurement	No output-locked comparison yet	Radar verifier swap	Spreadsheet dual scoring
Information integrity	Artifact forensic	Artifact forensic	Wrong-metadata and oracle controls

Figure 4: Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

Case	Result	Maximum claim
Invoice	$P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$	B is necessary and sufficient for the narrow same-contract effect.
Energy	Local mediation in the original task; necessity and sufficiency do not survive the variant	Local component effect, not transferable mediation.
SearchQA	9 repairs, 0 regressions; stable success 7/12, below the strong gate	Directional mediation on part of the held-out set.

Table 4: Artifact-level mediation evidence.

checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt artifact at 10/12, whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evaluator observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

From Reported Gains to Auditable Claims

Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

Audit step	Required test → maximum supported claim
Validity	Execution and verifier status plus assertion coverage → eligible observation
Retention	Frozen actual-parent/final fresh repeats → retained artifact effect
Behavioral scope	Entity, parameter, input, contract, or family variants → bounded controlled transfer
Evolution pathway	Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation
Measurement	Output-locked calibrated verifier comparison → measurement dependence
Information integrity	Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported

Table 5: Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

Related Work

Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. SkillsBench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks (Li et al. 2026; Han et al. 2026). SkillLearnBench evaluates continual skill generation (Zhong et al. 2026), while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops (Liu et al. 2026b; Li and Hu 2026; Yang et al. 2026; Xia et al. 2026; Zhang et al. 2026b). Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption (Huang et al. 2026); How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval (Liu et al. 2026a); SkillCoach evaluates process-level skill use (Zhu et al. 2026); and SkillJuror isolates the effect of skill organization (Chen et al. 2026).

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an agent trajectory (Zhou et al. 2026a). SEA-Eval evaluates self-evolving agents beyond isolated episodes (Jiang et al. 2026). Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal (Zhang et al. 2026b,c). More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability (Amodei et al. 2016; Oren et al. 2023).

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

Limitations

Our evidence is mechanism-focused rather than a benchmark-wide prevalence study. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. We therefore do not claim that any observed mechanism is universal or that one method outperforms another.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

Conclusion

We study whether benchmark gains from self-evolving persistent skills correspond to retained behavioral improvement and, independently, which lifecycle mechanisms produce those gains. In the recorded variant evaluations, SkillEvolver retains positive mean gains across changes in input representation, task-instance structure, and coupled requirements; SPARK shows gains on two types of change, while CoEvoSkills shows declines on all three. Execution traces connect successful cases to reusable diagnostic steps and preservation of task constraints. Our audits show that revised skill artifacts can produce real behavioral changes, but selected gains can disappear under fresh deployment and the transfer established so far remains narrow. Feedback, optimizer variation, validation selection, artifact content, execution variability, and verifier choice can each shape the observed endpoint.

A higher benchmark score should therefore be reported only at the strongest level supported by matched evidence: a selected gain, a retained artifact effect, bounded controlled transfer, evolution-pathway dependence, or measurement dependence. We provide a lifecycle audit that links each claim to the required intervention and reporting evidence. Claims of transferable skill learning require frozen parent-to-final improvement that survives fresh deployment and predeclared task variation under valid measurement; weaker evidence should be reported more narrowly.

References

Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; and Zhang, W. 2026. SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior. arXiv:2606.11543.

Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401.

Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; and Luo, C. 2026. From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills. arXiv:2605.23899.

Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; and Xiao, Y. 2026. SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment. arXiv:2604.08988.

Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; and Song, D. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670.

Li, Z.; and Hu, Y. 2026. SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History. arXiv:2606.08671.

Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; and Chang, S. 2026a. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. arXiv:2604.04323.

Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li, H.; and Song, Y. 2026b. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. arXiv:2606.01139.

Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; and Hashimoto, T. B. 2023. Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234.

Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; and Luo, C. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904.

Zhang, G.; Zhu, E.; Zhou, J.; Jia, C.; and Wang, H. 2026a. SkillEvolver: Skill Learning as a Meta-Skill. arXiv:2605.10500.

Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; and Yu, P. S. 2026b. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687.

Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026c. Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents. arXiv:2607.12790.

Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; and Xiong, C. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. arXiv:2604.20087.

Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; and Hu, X. 2026a. Counterfactual Trace Auditing of LLM Agent Skills. arXiv:2605.11946.

Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; and Metaxas, D. N. 2026b. Evidence Over Plans: Online Trajectory Verification for Skill Distillation. arXiv:2605.09192.

Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; and Yue, Y. 2026. SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use. arXiv:2607.01874.